

Module 11 Structured Note: Final Review, Assessment, and Capstone

1. What this final module is about

Module 11 is not a new isolated string algorithm.

It is the **final review and capstone module**.

That means this module brings together everything learned in Modules 1 to 10 and uses those ideas to build one complete program.

The main goal is:

Combine all string algorithms into one usable program and prove that you understand when and why each method works.

In earlier modules, each lesson focused on one skill.

Examples:

```
// Module 5
Reverse a string

// Module 7
Find duplicates

// Module 9
Check anagrams

// Module 10
Generate permutations
```

But in real programming, you often need to combine many smaller functions into one larger program.

So this final module focuses on building a **menu-driven String Lab**.

2. Current progress before Module 11

Before starting this final module, you have already completed Modules 1 to 10.

Here is what you have learned so far:

Area	Modules	Main skills
String foundations	1–2	<code>char</code> , ASCII, character arrays, <code>\0</code> , length, case conversion
Character counting	3–4	Count vowels, consonants, words, and validate strings
Two-pointer logic	5–6	Reverse strings, compare strings, check palindromes
Counting and hashing	7–9	Find duplicates, use <code>H[26]</code> , use bits, check anagrams
Recursion and backtracking	10	Generate permutations using flags, result arrays, and swapping

So your current level is the **integration stage**.

That means you are no longer just learning one small operation at a time.

Now you must learn how to organize all those operations into one complete program.

3. Big idea of the capstone

The capstone project is a **String Lab**.

A String Lab is a program that allows the user to choose different string operations from a menu.

Example menu:

```
--- String Lab ---
1. Find length
2. Toggle case
3. Count vowels and consonants
4. Count words
5. Validate string
6. Reverse string
7. Compare two strings
8. Check palindrome
9. Find duplicates
10. Check anagram
11. Generate permutations
0. Exit
```

The user enters a number.

Then the program runs the correct string algorithm.

Example:

```
Enter choice: 1
Length is 7
```

or:

```
Enter choice: 8
Palindrome
```

The main idea is:

Take input, show a menu, call the correct function, print the result, and explain the complexity.

4. What this final module tests

This final module checks whether you can do four important things.

4.1 Pick the correct algorithm

You should know which method to use for each problem.

Example:

Problem	Good method
Find length	Scan until <code>\0</code>
Reverse string	Extra array or two-pointer swap
Check palindrome	Two-pointer comparison
Find duplicates	Hash table or bitwise method
Check anagram	Frequency/counting array
Generate permutations	Recursion and backtracking

4.2 Write the algorithm without library shortcuts

For this course, you should practice writing the logic yourself.

For example, instead of using:

```
strlen(S);
```

You should write:

```
int length(char S[]) {  
    int i;  
  
    for (i = 0; S[i] != '\0'; i++) {  
    }  
  
    return i;  
}
```

This helps you understand how C-style strings really work.

4.3 Explain time and space complexity

For every operation, you should be able to say:

```
Time Complexity: O(...)  
Space Complexity: O(...)
```

Example:

```
Palindrome check  
Time Complexity: O(N)  
Space Complexity: O(1)
```

Why?

Because the algorithm scans the string from both ends and uses only a few variables.

4.4 Handle edge cases

An edge case is a special input that may break weak code.

Examples:

```
Empty string: ""
String with spaces: "How are you"
String with extra spaces: "How   are   you"
Mixed case: "NiTin"
Repeated letters: "banana"
Symbols: "An?1"
Digits: "abc123"
```

Good code should not only work for easy examples.

It should also handle unusual input safely.

5. Main habit for each algorithm

For every string algorithm, follow this habit:

1. Define the input.
2. Define the output.
3. Identify the stopping condition.
4. Dry-run indexes and array values.
5. Write the code.
6. State time complexity.
7. State space complexity.
8. Test normal and edge cases.

This habit proves that you understand the algorithm, not just the final code.

6. Main structure of the String Lab program

The String Lab should be organized using functions.

The `main()` function should not contain all the algorithm logic.

Instead, `main()` should:

1. Read input.
2. Show the menu.
3. Take the user's choice.
4. Call the correct function.

5. Print the result.
6. Repeat until the user exits.

Basic structure:

```
#include <stdio.h>

int main() {
    char A[100];
    int choice;

    printf("Enter a string: ");
    fgets(A, 100, stdin);

    do {
        printf("\n--- String Lab ---\n");
        printf("1. Find length\n");
        printf("2. Toggle case\n");
        printf("3. Count vowels and consonants\n");
        printf("4. Count words\n");
        printf("5. Validate string\n");
        printf("6. Reverse string\n");
        printf("7. Check palindrome\n");
        printf("8. Find duplicates\n");
        printf("9. Check anagram\n");
        printf("10. Generate permutations\n");
        printf("0. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                // call length function
                break;

            case 2:
                // call toggle function
                break;

            case 3:
                // call vowel/consonant counting function
                break;

            case 0:
                printf("Exiting...\n");
```

```

        break;

    default:
        printf("Invalid choice\n");
    }

} while (choice != 0);

return 0;
}

```

Beginner mental model:

`main()` is the controller. The actual work should be done by separate functions.

7. Important input note: `fgets()` and newline

In earlier modules, you may have used strings like this:

```
char A[] = "welcome";
```

But in a real program, you may read input using:

```
fgets(A, 100, stdin);
```

`fgets()` is safer than `gets()` because it knows the maximum array size.

But `fgets()` may store the newline character `\n`.

Example input:

```
hello
```

The array may contain:

```
h e l l o \n \0
```

So some functions should stop at either:

'\0'

or:

'\n'

Example:

```
for (i = 0; A[i] != '\0' && A[i] != '\n'; i++) {  
}
```

This prevents the newline from being treated as a normal character.

Part A: Core functions for the String Lab

8. Function 1: Find string length

8.1 Goal

Find the number of visible characters before the string ends.

For:

welcome

The length is:

7

because the visible characters are:

w e l c o m e

The null terminator `\0` is not counted.

8.2 Basic logic

Start from index 0.
Move forward one character at a time.
Stop when you reach '\0' or '\n'.
The stopping index is the length.

8.3 Code

```
int length(char S[]) {  
    int i;  
  
    for (i = 0; S[i] != '\0' && S[i] != '\n'; i++) {  
        // move forward  
    }  
  
    return i;  
}
```

8.4 Dry run

String:

welcome

Memory idea:

Index: 0 1 2 3 4 5 6 7
Value: w e l c o m e \0

Trace:

i	S[i]	Action
0	w	continue
1	e	continue
2	l	continue
3	c	continue

i	S[i]	Action
4	o	continue
5	m	continue
6	e	continue
7	\0	stop

Final answer:

Length = 7

8.5 Complexity

Time Complexity: $O(N)$
Space Complexity: $O(1)$

The string is scanned once, and only one index variable is used.

9. Function 2: Toggle case

9.1 Goal

Change:

Uppercase letters -> lowercase letters
Lowercase letters -> uppercase letters
Digits -> unchanged
Symbols -> unchanged
Spaces -> unchanged

Example:

Input: wE1CoMe5
Output: We1cOmE5

9.2 Required ASCII idea

Uppercase letters:

'A' to 'Z'

Lowercase letters:

'a' to 'z'

Difference between uppercase and lowercase letters:

32

So:

Uppercase + 32 = lowercase
Lowercase - 32 = uppercase

Examples:

'A' + 32 = 'a'
'a' - 32 = 'A'

9.3 Code

```
void toggleCase(char A[]) {  
    int i;  
  
    for (i = 0; A[i] != '\0' && A[i] != '\n'; i++) {  
        if (A[i] >= 'A' && A[i] <= 'Z') {  
            A[i] = A[i] + 32;  
        } else if (A[i] >= 'a' && A[i] <= 'z') {  
            A[i] = A[i] - 32;  
        }  
    }  
}
```

9.4 Important warning

Do not blindly add or subtract `32`.

Wrong:

```
A[i] = A[i] + 32;
```

Why is this wrong?

Because not every character is uppercase.

Example:

```
'1' + 32
```

This does not produce a useful letter.

Correct idea:

```
Check the range first.  
Then change only letters.
```

9.5 Complexity

```
Time Complexity: O(N)  
Space Complexity: O(1)
```

The function scans the string once and changes it in-place.

10. Function 3: Count vowels and consonants

10.1 Goal

Count:

Vowels
Consonants

Vowels are:

a e i o u
A E I O U

Consonants are alphabet letters that are not vowels.

Important:

Digits, spaces, and symbols are not consonants.

10.2 Code

```
void countVowelsConsonants(char A[]) {  
    int i;  
    int vcount = 0;  
    int ccount = 0;  
  
    for (i = 0; A[i] != '\0' && A[i] != '\n'; i++) {  
        if (A[i] == 'a' || A[i] == 'e' || A[i] == 'i' || A[i] == 'o' || A[i] ==  
'u' ||  
            A[i] == 'A' || A[i] == 'E' || A[i] == 'I' || A[i] == 'O' || A[i] ==  
'U') {  
            vcount++;  
        } else if ((A[i] >= 'A' && A[i] <= 'Z') ||  
                    (A[i] >= 'a' && A[i] <= 'z')) {  
            ccount++;  
        }  
    }  
  
    printf("Vowels: %d\n", vcount);  
    printf("Consonants: %d\n", ccount);  
}
```

10.3 Dry run

Input:

How are you! 123

Important characters:

Character	Type	Action
H	consonant	ccount++
o	vowel	vcount++
w	consonant	ccount++
space	not alphabet	ignore
a	vowel	vcount++
r	consonant	ccount++
e	vowel	vcount++
y	consonant	ccount++
o	vowel	vcount++
u	vowel	vcount++
!	symbol	ignore
1, 2, 3	digits	ignore

Final count:

Vowels = 5
Consonants = 4

10.4 Complexity

Time Complexity: $O(N)$
Space Complexity: $O(1)$

11. Function 4: Count words

11.1 Goal

Count how many words are in a string.

A simple word can be treated as:

A group of non-space characters.

Example:

How are you

Words:

How
are
you

Word count:

3

11.2 Problem with simple space counting

A simple formula is:

words = spaces + 1

This works for clean input:

How are you

But it fails with extra spaces:

How are you

There are many spaces, but still only three words.

So the better method is the `inWord` method.

11.3 Better logic using `inWord`

`inWord` tells us whether we are currently inside a word.

```
inWord = 0 means outside a word
inWord = 1 means inside a word
```

Logic:

```
If current character is not a space and inWord is 0:
    This is the start of a new word.
    word++
    inWord = 1

If current character is a space:
    We are outside a word.
    inWord = 0
```

11.4 Code

```
int countWords(char A[]) {
    int i;
    int word = 0;
    int inWord = 0;

    for (i = 0; A[i] != '\0' && A[i] != '\n'; i++) {
        if (A[i] != ' ' && inWord == 0) {
            word++;
            inWord = 1;
        } else if (A[i] == ' ') {
            inWord = 0;
        }
    }

    return word;
}
```

11.5 Complexity

Time Complexity: $O(N)$
Space Complexity: $O(1)$

12. Function 5: Validate a string

12.1 Goal

Check whether a string follows a rule.

Original validation rule:

Only letters and digits are allowed.

Allowed characters:

A to Z
a to z
0 to 9

Invalid characters:

? ! @ # _ space

unless the rule is changed to allow them.

12.2 Main idea

For every character:

If it is uppercase, it is valid.
Else if it is lowercase, it is valid.
Else if it is a digit, it is valid.
Else it is invalid.

One invalid character makes the whole string invalid.

12.3 Code

```
int validate(char A[]) {
    int i;

    for (i = 0; A[i] != '\0' && A[i] != '\n'; i++) {
        if (!(A[i] >= 'A' && A[i] <= 'Z') &&
            !(A[i] >= 'a' && A[i] <= 'z') &&
            !(A[i] >= '0' && A[i] <= '9')) {
            return 0;
        }
    }

    return 1;
}
```

12.4 Dry run

Input:

An?1

Trace:

Character	Result
A	valid uppercase
n	valid lowercase
?	invalid

The function immediately returns:

0

Final result:

Invalid String

12.5 Important rule about returning

Return `0` immediately when an invalid character is found.

But return `1` only after the full string has been checked.

Wrong idea:

```
If the first character is valid, return 1.
```

Why is that wrong?

Example:

```
A???
```

The first character is valid, but the full string is invalid.

Correct idea:

```
The string is valid only if every character is valid.
```

12.6 Complexity

```
Worst-case Time Complexity: O(N)  
Best-case Time Complexity: O(1)  
Space Complexity: O(1)
```

Worst case happens when the string is valid or the invalid character is at the end.

Best case happens when the first character is invalid.

13. Function 6: Reverse a string

There are two important methods.

Method 1: Reverse using an extra array
Method 2: Reverse in-place using two pointers

14. Reverse method 1: Extra array

14.1 Goal

Copy the original string backward into another array.

Example:

Original: python
Reverse: nohtyp

14.2 Code

```
void reverseExtraArray(char A[]) {  
    char B[100];  
    int i, j;  
  
    for (i = 0; A[i] != '\0' && A[i] != '\n'; i++) {  
    }  
  
    i = i - 1;  
  
    for (j = 0; i >= 0; i--, j++) {  
        B[j] = A[i];  
    }  
  
    B[j] = '\0';  
  
    printf("Reversed string: %s\n", B);  
}
```

14.3 Important point

You must add:

```
B[j] = '\0';
```

Why?

Because a C-style string must end with `\0`.

If `\0` is missing, `printf("%s", B)` may print garbage after the real reversed string.

14.4 Complexity

```
Time Complexity: O(N)
Space Complexity: O(N)
```

It uses another array, so extra space is `O(N)`.

15. Reverse method 2: In-place reversal

15.1 Goal

Reverse the same array directly without creating another string array.

Use two indexes:

```
i = left side
j = right side
```

Swap characters until the indexes meet or cross.

15.2 Code

```
void reverseInPlace(char A[]) {
    int i, j;
    char t;

    for (j = 0; A[j] != '\0' && A[j] != '\n'; j++) {
    }

    j = j - 1;
```

```
    for (i = 0; i < j; i++, j--) {  
        t = A[i];  
        A[i] = A[j];  
        A[j] = t;  
    }  
  
    printf("Reversed string: %s\n", A);  
}
```

15.3 Dry run

Input:

python

Initial:

p y t h o n

Swaps:

Swap p and n -> n y t h o p
Swap y and o -> n o t h y p
Swap t and h -> n o h t y p

Final:

nohtyp

15.4 Complexity

Time Complexity: $O(N)$
Space Complexity: $O(1)$

This method uses only a few variables, so it is more memory efficient.

16. Function 7: Compare two strings

16.1 Goal

Compare two strings character by character.

Possible results:

```
Strings are equal
First string is smaller
First string is greater
```

16.2 Main idea

Compare from left to right.

Stop when:

```
A mismatch is found
or
one string ends
```

The first different character decides the result.

16.3 Code

```
void compareStrings(char A[], char B[]) {
    int i, j;

    for (i = 0, j = 0;
        A[i] != '\0' && A[i] != '\n' && B[j] != '\0' && B[j] != '\n';
        i++, j++) {
        if (A[i] != B[j]) {
            break;
        }
    }

    if ((A[i] == B[j]) ||
        ((A[i] == '\0' || A[i] == '\n') && (B[j] == '\0' || B[j] == '\n')))) {
        printf("Strings are Equal\n");
    } else if (A[i] < B[j]) {

```

```
        printf("First string is Smaller\n");
    } else {
        printf("First string is Greater\n");
    }
}
```

16.4 Dry run

Input:

```
A = Painter
B = Painting
```

Compare:

Index	A	B	Result
0	P	P	same
1	a	a	same
2	i	i	same
3	n	n	same
4	t	t	same
5	e	i	different

ASCII values:

```
'e' = 101
'i' = 105
```

Since:

```
101 < 105
```

Final result:

```
First string is Smaller
```

16.5 Complexity

```
Time Complexity: O(N)
Space Complexity: O(1)
```

17. Function 8: Check palindrome

17.1 Goal

Check whether a string reads the same forward and backward.

Examples:

```
madam -> palindrome
level -> palindrome
hello -> not palindrome
```

17.2 Main idea

Use two pointers:

```
i = first character
j = last character
```

Compare:

```
A[i] with A[j]
```

Then move inward:

```
i++
j--
```

If any pair does not match, the string is not a palindrome.

17.3 Code

```
int isPalindrome(char A[]) {
    int i, j;

    for (j = 0; A[j] != '\0' && A[j] != '\n'; j++) {
    }

    j = j - 1;

    for (i = 0; i < j; i++, j--) {
        if (A[i] != A[j]) {
            return 0;
        }
    }

    return 1;
}
```

17.4 Dry run

Input:

madam

Compare:

Step	Left	Right	Result
1	m	m	same
2	a	a	same

Then the indexes meet at the middle character **d**.

No mismatch was found.

Final result:

Palindrome

17.5 Case sensitivity warning

By default:

```
'N' != 'n'
```

So:

```
Nitin
```

may not be treated as a palindrome unless you first convert it to one case.

Example:

```
Nitin -> nitin
```

Then it becomes a palindrome.

17.6 Complexity

```
Time Complexity: O(N)  
Space Complexity: O(1)
```

18. Function 9: Find duplicates

There are multiple ways to find duplicates.

The capstone should mainly review:

```
Hash table/counting array method  
Bitwise method
```

19. Duplicate method 1: Hash table/counting array

19.1 Goal

Find characters that appear more than once.

Example:

finding

Duplicates:

i
n

19.2 Code for lowercase letters

```
void findDuplicates(char A[]) {  
    int H[26] = {0};  
    int i;  
  
    for (i = 0; A[i] != '\0' && A[i] != '\n'; i++) {  
        if (A[i] >= 'a' && A[i] <= 'z') {  
            H[A[i] - 'a']++;  
        }  
    }  
  
    for (i = 0; i < 26; i++) {  
        if (H[i] > 1) {  
            printf("%c appears %d times\n", i + 'a', H[i]);  
        }  
    }  
}
```

19.3 Dry run

Input:

```
finding
```

Important mappings:

```
f - a = 5  
i - a = 8  
n - a = 13  
d - a = 3  
g - a = 6
```

Counts:

```
H[8] = 2 -> i  
H[13] = 2 -> n
```

Output:

```
i appears 2 times  
n appears 2 times
```

19.4 Complexity

```
Time Complexity: O(N)  
Space Complexity: O(1)
```

The array has fixed size 26, so it is treated as constant extra space.

20. Duplicate method 2: Bitwise operations

20.1 Goal

Use bits inside one integer to remember which letters have already appeared.

Each lowercase letter gets one bit:

```
a -> bit 0  
b -> bit 1  
c -> bit 2
```

```
...  
z -> bit 25
```

20.2 Main idea

For each letter:

```
Create a mask using left shift.  
Use AND to check if the bit is already ON.  
Use OR to turn the bit ON if it is not already ON.
```

20.3 Code

```
void findDuplicatesBitwise(char A[]) {  
    long int H = 0;  
    long int X = 0;  
    int i;  
  
    for (i = 0; A[i] != '\0' && A[i] != '\n'; i++) {  
        if (A[i] >= 'a' && A[i] <= 'z') {  
            X = 1;  
            X = X << (A[i] - 'a');  
  
            if ((X & H) > 0) {  
                printf("%c is duplicate\n", A[i]);  
            } else {  
                H = X | H;  
            }  
        }  
    }  
}
```

20.4 Important limitation

The bitwise method tracks only presence.

It can answer:

Has this letter appeared before?

But it cannot directly answer:

How many times did this letter appear?

So if exact counts are needed, use the hash table method.

20.5 Complexity

Time Complexity: $O(N)$
Space Complexity: $O(1)$

21. Function 10: Check anagram

21.1 Goal

Check whether two strings contain the same letters with the same frequencies.

Example:

decimal
medical

They are anagrams because they contain the same letters.

21.2 Main rule

Anagrams require:

Same length
Same letters
Same letter counts

Order does not matter.

Counts do matter.

Example:

```
aab  
abb
```

These are not anagrams because:

```
aab -> a appears 2 times, b appears 1 time  
abb -> a appears 1 time, b appears 2 times
```

21.3 Code

```
int isAnagram(char A[], char B[]) {  
    int H[26] = {0};  
    int i;  
  
    if (length(A) != length(B)) {  
        return 0;  
    }  
  
    for (i = 0; A[i] != '\0' && A[i] != '\n'; i++) {  
        if (A[i] >= 'a' && A[i] <= 'z') {  
            H[A[i] - 'a']++;  
        } else {  
            return 0;  
        }  
    }  
  
    for (i = 0; B[i] != '\0' && B[i] != '\n'; i++) {  
        if (B[i] >= 'a' && B[i] <= 'z') {  
            H[B[i] - 'a']--;  
  
            if (H[B[i] - 'a'] < 0) {  
                return 0;  
            }  
        } else {  
            return 0;  
        }  
    }  
  
    return 1;  
}
```

21.4 Dry run

Input:

```
A = decimal
B = medical
```

Step 1: Count letters from `decimal`.

```
d, e, c, i, m, a, l each get count 1
```

Step 2: Subtract letters from `medical`.

```
m -> count becomes 0
e -> count becomes 0
d -> count becomes 0
i -> count becomes 0
c -> count becomes 0
a -> count becomes 0
l -> count becomes 0
```

No count becomes negative.

Final result:

```
It is an Anagram
```

21.5 Why bitwise is not enough for anagrams

Bitwise presence only checks whether a letter exists.

But anagrams need exact counts.

Example:

```
A = aab
B = abb
```

Both contain `a` and `b`, but the counts are different.

So bitwise presence may give the wrong answer.

For anagrams, use a counting array like:

```
int H[26] = {0};
```

21.6 Complexity

```
Time Complexity: O(N)
Space Complexity: O(1)
```

The `H[26]` array is fixed size.

22. Function 11: Generate permutations

22.1 Goal

Generate all arrangements of the characters in a string.

Example:

```
ABC
```

Permutations:

```
ABC
ACB
BAC
BCA
CAB
CBA
```

22.2 Number of permutations

If a string has length `n`, the number of permutations is:

$n!$

Examples:

```
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
```

This grows very quickly.

So permutation algorithms can become expensive for long strings.

23. Permutation method 1: Flag array and result array

23.1 Main idea

Use:

```
flag[] to remember which characters are already used
res[] to build the current permutation
k to track the current position
```

23.2 Code

```
void perm(char s[], int k) {
    static int flag[10] = {0};
    static char res[10];
    int i;

    if (s[k] == '\0') {
        res[k] = '\0';
        printf("%s\n", res);
    } else {
        for (i = 0; s[i] != '\0'; i++) {
            if (flag[i] == 0) {
                res[k] = s[i];
                flag[i] = 1;
                perm(s, k + 1);
            }
        }
    }
}
```

```
        flag[i] = 0;
    }
}
}
```

23.3 Backtracking idea

The important pattern is:

```
Choose a character.
Mark it as used.
Recurse to fill the next position.
Unmark it after returning.
Try another character.
```

The line:

```
flag[i] = 0;
```

is the backtracking step.

It undoes the choice so the character can be used in another branch.

24. Permutation method 2: In-place swapping

24.1 Main idea

Rearrange the original string by swapping characters.

Use:

```
l = current position
h = last index
```

At each position, try every possible character by swapping it into that position.

24.2 Code

```
void swap(char *x, char *y) {
    char temp = *x;
    *x = *y;
    *y = temp;
}

void permSwap(char s[], int l, int h) {
    int i;

    if (l == h) {
        printf("%s\n", s);
    } else {
        for (i = l; i <= h; i++) {
            swap(&s[l], &s[i]);
            permSwap(s, l + 1, h);
            swap(&s[l], &s[i]);
        }
    }
}
```

24.3 Why the second swap is important

This line after recursion is required:

```
swap(&s[l], &s[i]);
```

It restores the string to its previous state.

Without this second swap, the next branch may start with the wrong arrangement.

This is the backtracking step for the swapping method.

24.4 Complexity

```
Time Complexity:  $O(n \times n!)$ 
Space Complexity:  $O(n)$ 
```

Why time is high:

There are $n!$ permutations, and printing each permutation takes $O(n)$ time.

Part B: What the capstone should print

25. Required output style

For each operation, the capstone should print:

```
Result
Time Complexity
Space Complexity
Main idea
```

Example for palindrome:

```
Result: Palindrome
Time Complexity: O(N)
Space Complexity: O(1)
Main idea: Compare the first and last characters, then move inward.
```

This is important because the capstone is not only about getting the correct answer.

It is also about explaining how the answer was found.

26. Example capstone output

Input:

```
madam
```

Choice:

```
7. Check palindrome
```

Output:

Result: Palindrome
Time Complexity: $O(N)$
Space Complexity: $O(1)$
Main idea: Compare characters from both ends and move inward.

27. Complexity summary table

Operation	Main method	Time complexity	Extra space
Find length	Scan until <code>\0</code>	$O(N)$	$O(1)$
Toggle case	ASCII range checks	$O(N)$	$O(1)$
Count vowels/consonants	Character classification	$O(N)$	$O(1)$
Count words	<code>inWord</code> method	$O(N)$	$O(1)$
Validate string	Allowed range checks	$O(N)$ worst case	$O(1)$
Reverse with extra array	Copy backward	$O(N)$	$O(N)$
Reverse in-place	Two-pointer swap	$O(N)$	$O(1)$
Compare strings	Left-to-right comparison	$O(N)$	$O(1)$
Check palindrome	Two pointers	$O(N)$	$O(1)$
Find duplicates with hash table	<code>H[26]</code> count array	$O(N)$	$O(1)$
Find duplicates with bits	Bit mask	$O(N)$	$O(1)$
Check anagram	Frequency counting	$O(N)$	$O(1)$
Generate permutations	Backtracking	$O(n \times n!)$	$O(n)$

Part C: Common mistakes in the final module

28. Mistake 1: Forgetting `\0`

Wrong idea:

A string ends when the array capacity ends.

Correct idea:

A C-style string ends when `'\0'` is found.

Most loops should use:

```
A[i] != '\0'
```

29. Mistake 2: Ignoring the newline from `fgets()`

If you use `fgets()`, the string may include `\n`.

So some loops should stop at:

```
A[i] != '\0' && A[i] != '\n'
```

30. Mistake 3: Putting all code inside `main()`

Wrong style:

```
main() contains every algorithm directly.
```

Better style:

```
main() shows the menu and calls functions.  
Each string operation has its own function.
```

This makes the program easier to read, test, and fix.

31. Mistake 4: Returning valid too early

In validation, this is wrong:

```
First character is valid, so the whole string is valid.
```

Example:

```
A???
```

The first character is valid, but the full string is not.

Correct rule:

```
Return invalid immediately when one invalid character is found.  
Return valid only after checking the full string.
```

32. Mistake 5: Counting symbols as consonants

Wrong idea:

```
If it is not a vowel, it is a consonant.
```

Correct idea:

```
If it is a vowel, count vowel.  
Else if it is an alphabet letter, count consonant.  
Else ignore it.
```

Symbols, spaces, and digits are not consonants.

33. Mistake 6: Using `H[A[i] - 'a']` without checking the character

This is safe only for lowercase letters:

```
H[A[i] - 'a']++;
```

If `A[i]` is uppercase, digit, space, or symbol, the index may become invalid.

Safer version:

```
if (A[i] >= 'a' && A[i] <= 'z') {  
    H[A[i] - 'a']++;  
}
```

34. Mistake 7: Using bitwise method for exact anagram counts

Bitwise presence is not enough for anagrams.

Example:

```
aab  
abb
```

Both have `a` and `b`, but the counts are different.

So use a frequency array for anagrams.

35. Mistake 8: Forgetting to backtrack in permutations

In the flag method, you must undo the flag:

```
flag[i] = 0;
```

In the swapping method, you must swap back:

```
swap(&s[l], &s[i]);
```

Without backtracking, the algorithm will not explore all branches correctly.

Part D: Final checklist

36. Skills you should now have

You are ready to finish the Strings course if you can do these things without help:

1. Explain why C strings stop at '\0'.
2. Find string length without strlen().
3. Convert case using ASCII range checks.
4. Count vowels and consonants correctly.
5. Count words even with extra spaces.
6. Validate a string using allowed character ranges.
7. Reverse using an extra array.
8. Reverse in-place using two pointers.
9. Compare two strings character by character.
10. Check palindrome using two pointers.
11. Find duplicates using H[26].
12. Find duplicates using bitwise operations.
13. Check anagrams using frequency counts.
14. Generate permutations using backtracking.
15. State time and space complexity for each algorithm.

37. Final mental model

The final module is about moving from separate lessons to one complete project.

Earlier modules taught individual tools.

This module teaches you how to organize those tools into a usable program.

The final pattern is:

```
Take input.  
Show menu.  
Call the correct function.  
Print the result.  
Print complexity.  
Explain the main idea.  
Test edge cases.
```

That is the point where you move from:

```
I learned string algorithms.
```

to:

I can build a string-processing program.

Part E: Practice tasks for Module 11

38. Practice task 1: Build the menu

Create a menu with these options:

1. Length
2. Toggle case
3. Count vowels and consonants
4. Count words
5. Validate
6. Reverse
7. Palindrome
8. Duplicates
9. Anagram
10. Permutations
0. Exit

The menu should repeat until the user chooses .

39. Practice task 2: Add complexity output

After every operation, print:

```
Time Complexity: ...  
Space Complexity: ...
```

Example:

```
Length is 7  
Time Complexity: O(N)  
Space Complexity: O(1)
```

40. Practice task 3: Test edge cases

Test your String Lab with:

```
""  
"a"  
"madam"  
"Nitin"  
"How are you"  
"An?1"  
"finding"  
"decimal" and "medical"  
"aab" and "abb"  
"ABC"
```

For each test, write what result you expect before running the program.

41. Practice task 4: Explain one algorithm aloud

Pick one operation, such as palindrome checking.

Explain it using this format:

```
Input: A string  
Output: Palindrome or Not Palindrome  
Main idea: Compare characters from both ends  
Stopping condition: Stop when indexes meet or cross  
Time Complexity: O(N)  
Space Complexity: O(1)
```

This is how you prove that you understand the algorithm.

Final summary

Module 11 is the final review and capstone module.

It does not introduce a completely new string topic.

Instead, it asks you to combine all previous topics into one String Lab program.

The most important ideas are:

- Use functions.
- Use a menu.
- Call the correct algorithm.
- Handle edge cases.
- Print complexity.
- Explain the logic.

Once you can do this, you have completed the strings learning path at a beginner-friendly but solid level.